

XAMS Slow Control — Software Design Specification

Version 0.1 — 16 September 2026

Companion to `XAMS-slow-control-options.pdf` (the decision document). That document explains *why*; this one specifies *what to build*.

Status of this document: authoritative for implementation. Anything marked **TBD** must be resolved before the affected component is written; nothing else should be invented at implementation time. If a TBD blocks progress, record the assumption in the code comment and list it in §16.

1. Scope and principles

In scope. Monitoring and convenience control of the XAMS slow-control hardware: one NI cDAQ-9174 chassis (20 connected channels of 40 available), two CAEN DT1470ET HV supplies (8 channels), one Lake Shore 335, one UPS. Storage, plotting, alarming, and a small web UI.

Out of scope. Safety interlocks. Anything that protects hardware or people belongs in hardware or in the instrument's own limits. See §10.

Principles, in priority order.

1. **Readable beats clever.** The system is replacing a project that failed on maintainability, not on capability. A new student must be able to read a driver in one sitting.
 2. **Configuration is data, not code.** Channels, calibrations and limits live in `channels.yaml`, under version control. Adding a sensor touches no Python.
 3. **Monitoring never blocks on control.** A hung write must not stall a read loop.
 4. **Fail loudly, never silently.** A device that cannot be identified, a value that is stale, a service that has stopped producing — all raise alarms. A frozen plausible value is worse than a gap.
 5. **Restart is a no-op on hardware.** Starting a service never changes an instrument's state.
-

2. Repository layout

```
xams-sc/
├── README.md
├── DESIGN.md
├── pyproject.toml
├── config/
│   ├── channels.yaml
│   ├── devices.yaml
│   ├── alarms.yaml
│   ├── recipients.yaml
│   ├── secrets.yaml
│   └── secrets.example.yaml
├── src/xams_sc/
│   ├── __init__.py
│   ├── bus.py
│   ├── config.py
│   ├── model.py
│   ├── service.py
│   ├── scaling.py
│   ├── devices/
│   │   ├── cdaq.py
│   │   ├── derived.py
│   │   ├── caen.py
│   │   ├── lakeshore.py
│   │   └── ups.py
│   ├── sinks/
│   │   ├── pg_writer.py
│   │   ├── jsonl_writer.py
│   │   └── mongo_writer.py
│   ├── alarms/
│   │   ├── engine.py
│   │   └── notify.py
│   ├── api/
│   │   ├── app.py
│   │   └── client.py
│   ├── cli/
│   │   └── xams_ctl.py
│   ├── sql/
│   │   └── schema.sql
│   ├── grafana/
│   │   ├── provisioning/
│   │   │   ├── datasources/
│   │   │   └── dashboards/
│   │   └── dashboards/*.json
│   ├── procedures/*.yaml
│   ├── services/
│   ├── tests/
│   │   ├── test_scaling.py
│   │   ├── test_config.py
│   │   └── test_caen_protocol.py
│   └── tools/
│       ├── jsonl_to_parquet.py
│       ├── import_labview_csv.py
│       └── compare_to_labview.py
```

this document

channel definitions – single source of truth

device connection + identity

thresholds and severity routing

who gets notified – edited from the web UI

credentials – NOT in git (§11)

template with empty values, in git

MQTT publish/subscribe wrapper

load + validate YAML, compute config hash

Measurement, Status, Quality dataclasses

BaseService: lifecycle, reconnect, heartbeat

raw -> engineering units

flow integrator (§7.5)

PostgreSQL

temporary, see §9.5

threshold evaluation, hysteresis, dedup

SMS, email, sound

FastAPI: setpoints, status, web UI (127.0.0.1 only)

Python client for scripts and notebooks

start/stop/status for all services

tables + indexes – run on every install

points Grafana at PostgreSQL

tells Grafana where the JSONL lives

the dashboards themselves, in git

named control sequences (§10)

NSSM install scripts, one per service

parser tests against recorded responses

nightly conversion

convert CSV history into JSONL/Parquet

validation against historical CSV

Each service is a thin `__main__` that constructs a device class and runs `BaseService`. No business logic in entry points.

2.1 Why a message bus

Every service publishes to MQTT; nothing writes to a database directly. The cost is one extra moving part. What it buys:

Consumers can be added without touching a driver. Each of these is a subscriber, and none of them requires a change anywhere else: the PostgreSQL writer, the JSONL archive, the MongoDB writer during the transition (§9.5), the alarm engine, the flight recorder, the web UI, and a forwarder to a remote server if remote viewing is ever wanted.

Storage decisions stay reversible. This design moved from InfluxDB to PostgreSQL during its own drafting. Without a bus that would have meant a different database client in every driver; with one it is a single new file in `sinks/`.

Debugging is direct. `mosquitto_sub -t 'xams/#'` shows every measurement live, with no UI and no database involved.

It enforces a boundary. This is the real argument. Without a bus, "just write this one thing here as well" gradually accumulates inside the drivers, and acquisition, scaling, storage and alarming end up entangled — which is precisely what happened to the LabVIEW project. The bus makes the wrong thing awkward to do.

On MQTT specifically. It is an OASIS standard, also ISO/IEC 20922, with a frozen specification. It was built at IBM in 1999 for SCADA telemetry over low-bandwidth satellite links, so its design target is exactly this: instrument readings sent to a central point, tolerant of interruptions. Mosquitto (the broker) and paho-mqtt (the Python client) are both Eclipse Foundation projects, packaged in every major distribution, stable for over a decade. Neither is a one-person project that could disappear.

Note for context: MQTT is mainstream in industry and IoT, but it is *not* the convention in accelerator and large-detector controls, where EPICS and Tango dominate. For XAMS that is immaterial; it would matter only if the system ever had to integrate with a large controls group.

The cost, stated honestly. The broker becomes a single point through which all data flows. If it stops, services keep reading hardware but nothing is stored. Mitigations, all required (§6.4): the broker runs on loopback with automatic restart, and each service buffers its measurements in memory for several minutes and republishes them when the broker returns. A short broker outage then costs nothing; a long one costs a gap that is visible in the data and raised as an alarm, rather than passing unnoticed.

3. Naming conventions

Channel names are the identity of a measurement, used in MQTT topics, JSONL records, the database and the UI. They must be stable forever — renaming breaks history.

Use the instrument tag names the lab already uses, lowercased. `TT201` becomes `tt201`. These are ISA-style tags (TT = temperature transmitter, P = pressure, FM = flow meter) that match the plant numbering, so they carry meaning that an invented scheme would discard. An earlier draft of this document proposed names like `t_cathode_top`; that is superseded.

Rules: lowercase, ASCII, no separator inside a tag, `snake_case` only where a tag has no established form.

p101, p102, p103, p104	pressures (9207 voltage)
pmain	detector pressure
fm101	flow meter
tt103, tt104	temperatures, 1xx series
tt201 ... tt207	temperatures, 2xx series (PT1000)
tt301 ... tt304	temperatures, 3xx series
ttamb	ambient temperature
hv_anode, hv_cathode, hv_gate	HV channels
hv_pmt_top, hv_pmt_bot, hv_ts, hv_bs, hv_nai	
ls_sensor_a, ls_sensor_b	Lake Shore inputs
ls_heater_1, ls_heater_2	Lake Shore outputs
ups_*	UPS status

The `legacy` key in `channels.yaml` records the original casing (`TT201`, `Pmain`) so historical CSV data can still be matched.

The numbering appears to group by subsystem — `p101`–`p104` and `tt103`–`tt104` share the 1xx series, `tt201`–`tt207` the 2xx, `tt301`–`tt304` the 3xx. **TBD:** confirm what each series denotes, and record it in the `description` field of every channel. A tag without a location is only half an identity.

MQTT topics.

xams/meas/<channel>	measurement, JSON payload, retained
xams/status/<service>/heartbeat	ISO-8601 UTC timestamp, retained
xams/status/<service>/state	starting running degraded stopped
xams/alarm/<channel>	alarm state changes
xams/cmd/<device>/<action>	control requests (see §10)
xams/ack/<device>/<action>	command acknowledgements

Retained messages on `meas` and `status` mean a newly started subscriber immediately sees the current state instead of waiting for the next cycle.

4. Configuration

4.1 `devices.yaml`

```
cdaq:
  chassis: cDAQ1
  serial: "020C5E1C"
  modules:
    - alias: "9207"      model: NI9207  slot: 1  serial: "020DFD57"
    - alias: "9216_1"    model: NI9216  slot: 2  serial: "020F64D1"
    - alias: "9216_2"    model: NI9216  slot: 3  serial: "020F64D0"
    - alias: "9226"      model: NI9226  slot: 4  serial: "02159CBB"

caen:
  - id: hv_1
    match: {vid: "0403", pid: "6001", serial: "TBD"} # see §6.2
    board_address: 0
    board_serial: "TBD" # verified via BDSNUM on every connect
  - id: hv_2
    match: {vid: "0403", pid: "6001", serial: "TBD"}
    board_address: 1
    board_serial: "TBD"

lakeshore:
  match: {vid: "TBD", pid: "TBD", serial: "TBD"}
  baud: 57600 # TBD - confirm against instrument setting

ups:
  model: TBD
  connection: TBD
```

Device resolution never uses a COM number. It matches on USB hardware ID via `serial.tools.list_ports`, then confirms identity by querying the instrument (§6.2).

4.2 `channels.yaml`

One entry per channel. Schema:

Key	Required	Meaning
<code>name</code>	yes	channel identity (§3)
<code>device</code>	yes	<code>cdaq</code> / <code>hv_1</code> / <code>hv_2</code> / <code>lakeshore</code> / <code>ups</code>
<code>phys</code>	yes	physical address, e.g. <code>9207/ai5</code> , or HV channel index
<code>kind</code>	yes	<code>voltage</code> / <code>current</code> / <code>rtd</code> / <code>hv_vmon</code> / <code>hv_imon</code> / <code>temperature</code> / <code>status</code>
<code>unit</code>	yes	engineering unit string
<code>offset</code>	no	default 0.0
<code>multiplier</code>	no	default 1.0
<code>rtd</code>	for kind: <code>rtd</code>	<code>{type, r0, wiring}</code>

Key	Required	Meaning
legacy	no	name in the LabVIEW system
enabled	no	default true
description	no	free text: what and where
log_minmax	no	default false; store interval min/max as well (§9.1)

Example, using values recovered from the LabVIEW front panel:

```
- name: pmain
  device: cdaq
  phys: 9207/ai5
  kind: voltage
  offset: 0.0
  multiplier: 0.714
  unit: TBD           # front panel shows "Detector Pressure 1.53" without a unit
  legacy: Pmain
  description: detector pressure

- name: p101
  device: cdaq
  phys: 9207/ai0
  kind: voltage
  offset: 1.0
  multiplier: 25.0
  unit: TBD
  legacy: P101

- name: fm101
  device: cdaq
  phys: 9207/ai7
  kind: voltage
  offset: 0.0
  multiplier: 6.0
  unit: g/min         # mass flow. NOT SLPM – see §4.2 note below
  legacy: FM101

- name: tt301
  device: cdaq
  phys: 9216_1/ai0
  kind: rtd
  rtd: {type: PT3851, r0: 100.0, wiring: 3}
  unit: C
  legacy: TT301
  description: TBD – physical location

- name: tt201
  device: cdaq
  phys: 9226/ai0
  kind: rtd
  rtd: {type: PT3750, r0: 1000.0, wiring: 3}
  unit: C
  legacy: TT201
  description: TBD – physical location
```

Unconnected channels are listed explicitly with `enabled: false` rather than omitted, so the channel map is complete and nobody has to wonder later whether a gap is an oversight:

```
- name: spare_9216_2_ai0
  device: cdaq
  phys: 9216_2/ai0
  kind: rtd
  rtd: {type: PT3851, r0: 100.0, wiring: 3}
  unit: C
  enabled: false
  description: not connected
```

****Scaling is defined as `value = (raw - offset) * multiplier`.**** This matches the LabVIEW implementation exactly — the front panel labels the two columns "Offsets subtracted" and "Multipliers", in that order. Do not change the convention, or historical comparisons break.

Units are not recovered, and one of them in the old system is wrong. The LabVIEW front panel shows the scaling factors but not the engineering units — except for the flow, which it labels "Flow (SLPM)". **That label is incorrect: `fm101` is a mass flow in g/min.** It is recorded here explicitly so that nobody later "corrects" it back by consulting the old front panel.

This is a concrete instance of the risk that milestone 3 exists to catch: the number was right, the name was wrong, and nothing in the system noticed for years.

The pressure units are therefore **TBD** (§16) and must not be guessed. A value of 1.53 is equally plausible in bar or in another unit, and a wrong label propagates into every plot, every alarm threshold and eventually into a paper.

4.3 alarms.yaml

```
defaults:
  hysteresis: 0.02      # fraction of range, prevents chatter
  min_repeat_minutes: 15 # re-notification interval
  stale_after_seconds: 60

channels:
  pmain:
    high: {value: 1.8, severity: minor}
    hihi: {value: 2.0, severity: major, notify: [sms, email, sound]}
    low:  {value: 0.9, severity: minor}

routing:
  sms: {recipients: enabled} # everyone enabled in recipients.yaml
  email: {recipients: enabled}
```

Thresholds are **TBD** pending the `Error` and `Alarm` tab of the LabVIEW front panel.

4.4 recipients.yaml

Who receives alarm notifications. Separate from `alarms.yaml` because it changes often and for different reasons: people join the group, go on holiday, change numbers.

```
recipients:
- name: Auke-Pieter Colijn
  phone: "+31..."
  email: a.p.colijn@nikhef.nl
  enabled: true
- name: ...
  phone: "+31..."
  enabled: false      # temporarily off, without losing the number
```

Everyone with `enabled: true` receives the notification. No shift roster, no escalation chain: the list is the list.

Edited from the web UI — add, remove, or toggle `enabled` — and applied without restarting anything: the alarm engine watches the file and reloads it on change. Editing the file by hand works equally well and has the same effect.

This is deliberately unlike the alarm thresholds (§4.3), which stay in git and are applied with `xamsctl reload`. Changing a threshold silently alters what the system protects against and deserves review; changing a recipient does not.

Every change is appended to the same audit log as control actions (§10), so it remains recoverable who was on the list when a given alarm fired.

An empty list is a warning. If every recipient is disabled, alarms reach nobody. The alarm engine raises a low-severity alarm on startup and on reload when no recipient is enabled, so this is found on a quiet afternoon rather than during an incident.

4.5 Config hash

`config.py` computes a SHA-256 over the parsed, normalised contents of `channels.yaml`, `devices.yaml` and `alarms.yaml` (not `recipients.yaml`, which does not affect the data) and exposes the first 7 hex characters as `config_hash`. It is written into every JSONL daily file header (§9.3) and published on `xams/status/config`.

5. Data model

```
@dataclass(frozen=True)
class Measurement:
    t: datetime          # timezone-aware, UTC
    channel: str
    value: float          # engineering units
    unit: str
    raw: float | None = None
    quality: Quality = Quality.OK

class Quality(Enum):
    OK = "ok"
    STALE = "stale"          # not refreshed within stale_after_seconds
    ERROR = "error"          # read failed
    UNVERIFIED = "unverified" # device identity not confirmed
```

MQTT payload on `xams/meas/<channel>`:

```
{"t":"2026-09-16T13:02:00.000Z","ch":"pmain","v":1.530,"raw":2.143,"u":"bar","q":"ok"}
```

Timestamps are UTC, ISO-8601, millisecond precision, taken **at the moment of the hardware read**, not at publish time.

`raw` is written for the first year of operation at minimum (§9.4).

6. Service contract

Every device service inherits `BaseService` and must satisfy all of the following. This is the specification; deviations need a comment explaining why.

6.1 Lifecycle

```
resolve device → verify identity → configure → loop { read, publish, heartbeat }
                ↓ fail
                log FATAL, publish state=stopped, exit non-zero
```

- **Startup is read-only.** No setpoint is written, no channel enabled, no state restored. A restart must be invisible to the hardware.
- **A busy device is a clean failure**, not a stack trace: `FATAL: cDAQ1 in use by another process (LabVIEW running?)`. Refusing to start.
- **One instance only.** A named mutex (Windows) or lock file prevents a second copy.
- **Reconnect loop.** On a read failure: publish `quality=error`, back off (1 s, 2 s, 4 s ... capped at 30 s), retry indefinitely. The service never exits because a device disappeared; it exits only when it cannot be identified at startup.

6.2 Identity verification

Applies to every serial device, and is mandatory before any write.

A device is identified by **asking it who it is**, never by which COM port it happens to occupy.

1. Enumerate ports with `serial.tools.list_ports`, narrowing by VID/PID and — where available — USB serial number.
2. Open each candidate and query the instrument for its own identity:
 - CAEN: `$BD:<addr>,CMD:MON,PAR:BDSNUM` and `PAR:BDNAME`
 - Lake Shore: `*IDN?`
1. Compare against `devices.yaml`.

A COM port renumbered by Windows, a device moved to another hub socket, or the two supplies' cables swapped are therefore all harmless: the service finds each unit wherever it is. This holds even if the FTDI chips carry no unique USB serial numbers, because step 2 asks the instrument itself.

This replaces a real weakness of the present system, where COM8 and COM5 are fixed choices on the LabVIEW front panel. If Windows renumbers them, someone must notice and correct it by hand — and if the two supplies exchange numbers, LabVIEW will talk to the wrong one without any error, since both are valid CAEN units giving well-formed replies.

Behaviour on the three outcomes:

board_serial in devices.yaml	Behaviour
TBD (not yet known)	starts read-only ; publishes with <code>quality=unverified</code> ; every write refused
set, and it matches	normal operation; writes permitted
set, and it does not match	refuse to start, raise an alarm, do not guess

The first row exists so the configuration can bootstrap itself. The serial numbers are not needed in order to communicate — they are needed in order to *write safely*. Connect the service, let it report what it found, record that in `devices.yaml`, and the channel moves from `unverified` to `ok`. Monitoring (milestone 5) can therefore run before the serials are known, with the data visibly marked as unverified rather than silently trusted.

6.3 Heartbeat and staleness

- Publish `xams/status/<service>/heartbeat` every cycle.
- A separate monitor raises an alarm when a heartbeat exceeds `stale_after_seconds`.
- A channel not refreshed within the same window is republished with `quality=stale`. **Never leave a stale value standing as if fresh.**

6.4 Broker outage

The MQTT broker is a shared dependency (§2.1). Every service therefore:

- buffers measurements in a bounded in-memory queue (default 5 minutes' worth) while the broker is unreachable;
- republishes the buffer, in order, once the broker returns;
- raises an alarm if the buffer overflows, and records the gap explicitly rather than letting it pass silently;
- **keeps reading the hardware throughout.** A broker outage must never stop acquisition.

The broker itself runs on loopback under NSSM with automatic restart, so the common case is an outage of a few seconds.

7. Device specifications

7.1 cDAQ service (`devices/cdaq.py`)

Read-only. The chassis has no output module, so this service has no control path.

Tasks. All four modules are low-rate delta-sigma with differing aggregate rates and cannot share one hardware-timed task. Slow control reads at 1 Hz (§9.1), so use **on-demand (software-timed) reads**, one task per module, polled in sequence. The CompactDAQ timing-engine constraint then does not apply.

Task	Module alias	Channels	DAQmx call
1	9207	ai0:7 voltage	add_ai_voltage_chan
1b	9207	ai8:15 current	add_ai_current_chan — TBD: confirm these are used at all
2	9216_1	ai0:6	add_ai_rtd_chan
3	9216_2	—	entirely unconnected; task not created
4	9226	ai0:6	add_ai_rtd_chan

NI 9207 voltage channels (recovered from the LabVIEW front panel). Applied value is $(\text{raw} - \text{offset}) * \text{multiplier}$:

Channel	Tag	Offset	Multiplier	Role
9207/ai0	p101	1	25	
9207/ai1	p102	0	1	
9207/ai2	p103	0	1	
9207/ai3	p104	0	1	
9207/ai4	v4	0	1	generic name — likely unused
9207/ai5	pmain	0	0.714	detector pressure
9207/ai6	v6	0	1	generic name — likely unused
9207/ai7	fm101	0	6	flow meter

Current channels ai8:15 are named i0 – i7 with offset 0 and multiplier 1 throughout, which suggests they are unused (§16).

RTD channel map (from the lab, September 2026):

Module	Type	Channel	Tag	Notes
9226	PT1000	ai0 ... ai6	tt201 ... tt207	seven consecutive
9226	PT1000	ai7	—	not connected
9216_1	PT100	ai0	tt301	
9216_1	PT100	ai1	tt302	
9216_1	PT100	ai2	tt103	
9216_1	PT100	ai3	tt104	
9216_1	PT100	ai4	ttamb	ambient
9216_1	PT100	ai5	tt303	
9216_1	PT100	ai6	tt304	
9216_1	PT100	ai7	—	not connected
9216_2	PT100	ai0 ... ai7	—	module entirely unconnected

Fourteen RTDs are in use, not twenty-four. The whole of 9216_2 is free, which means eight spare PT100 inputs are already wired into the chassis — room for expansion without buying hardware. Do not create a DAQmx task for it; list its

channels in `channels.yaml` with `enabled: false` so the map stays complete.

RTD configuration (confirmed from the LabVIEW front panel):

Module	r ₀	rtd_type	resistance_config
NI 9216 (both)	100.0	RTDType.PT_3851	THREE_WIRE
NI 9226	1000.0	RTDType.PT_3750	THREE_WIRE

No custom Callendar–Van Dusen coefficients are in use. Let DAQmx return °C directly; do not hand-roll the conversion.

```
task.ai_channels.add_ai_rtd_chan(  
    "9216_1/ai0:7",  
    rtd_type=RTDType.PT_3851,  
    resistance_config=ResistanceConfiguration.THREE_WIRE,  
    current_excit_source=ExcitationSource.INTERNAL,  
    r_0=100.0,  
    units=TemperatureUnits.DEG_C,  
)
```

Modules are addressed by **alias** (`9207/ai0`), not `cDAQ1Mod1/ai0`. Aliases are configured in NI-MAX and survive re-slotting.

Timing mode: High Speed, as currently configured. 50 Hz rejection comes from the module's ADC mode, not from software averaging.

For reference, the settings the LabVIEW system uses: a 2 s cycle, a moving average over 2 samples on the flow and over 20 samples on the detector pressure, and an integrated-flow calculation. The new system reads at 1 Hz and logs 10 s means (§9.1), so these are not carried over directly, but they indicate the smoothing the operators are used to seeing.

7.2 CAEN service (`devices/caen.py`)

Two instances, one per supply. ASCII protocol over the USB virtual COM port; no vendor library.

```
$BD:<addr>,CMD:MON,PAR:VMON,CH:<n>      read measured voltage  
$BD:<addr>,CMD:MON,PAR:IMON,CH:<n>      read measured current  
$BD:<addr>,CMD:MON,PAR:STAT,CH:<n>      channel status word  
$BD:<addr>,CMD:SET,PAR:VSET,CH:<n>,VAL:<v>  set voltage    (control path, §10)
```

Terminate with `\r\n`. Parse the response; a malformed or absent reply is a read error, never a silently substituted value.

Channel names, from the LabVIEW front panel: `PMT top`, `PMT bot`, `TS`, `BS`, `Cathode`, `Gate`, `Anode`, `NaI`. The anode is on **CAEN 2, channel 3**. Mapping of the remaining seven to supply and channel index: **TBD**.

Two behaviours exist in the LabVIEW code whose purpose is not yet established: `anode_timing.vi` and `DAISY_polarity_signs.vi`. **TBD** — determine whether they must be reproduced before the control path is built.

7.3 Lake Shore service (`devices/lakeshore.py`)

Official `lakeshore` package. COM port resolved by hardware ID, identity confirmed with `*IDN?`.

Read: two sensor inputs, two heater outputs, setpoint, heater range, PID. Current PID settings: P = 100, I = 20, D = 0.

Control path (§10): setpoint within instrument limits only. The instrument's own setpoint limit and heater range remain the authority.

7.4 UPS service (`devices/ups.py`)

TBD — model and connection unknown. Read-only status: line power present, battery level, time on battery, timestamp. Publish `ups_*` channels.

A power event is one of the few things that can end a run, so this is worth having even though it is the smallest service.

7.5 Derived channels — the flow integrator

The present system integrates the flow and offers a reset button (`Average_flow_x_time.vi`). The feature is carried over, as `devices/derived.py` : a bus consumer that subscribes to `fm101` and publishes `fm101_total` , stored and plotted like any other channel.

This is the only stateful component in the system. Everything else is restartable without consequence; an integrator is not, which raises three requirements that must not be skipped.

It survives a restart. After every publish the accumulator is written to disk together with the timestamp of the last sample processed, and read back on startup. A Windows update that reset the total to zero would make the feature useless.

Gaps are recorded, never invented. If the service was down for two hours, what flowed during those two hours is unknown. Extrapolating the last value is tempting and wrong. Such intervals are excluded from the total and counted separately:

```
{"ch":"fm101_total","v":4213.8,"u":"g","q":"ok","gaps_s":7200}
```

The total then carries the evidence that it is an underestimate, instead of that having to be reconstructed months later. Samples whose `quality` is not `ok` are likewise not integrated.

Reset closes a period; it does not erase. Instead of zeroing a counter, the running period is closed and a new one opened, in a `flow_periods` table:

start	stop	total_g	gaps_s	reset_by
2026-08-01 09:14	2026-09-16 11:02	4213.8	0	apc
2026-09-16 11:02	—	118.4	0	—

This keeps the history of how much passed through during each period, rather than a number somebody once discarded. A reset is a control action and goes through the audit log (§10).

Units. `fm101` is a **mass flow in g/min** — *per minute*. The integral is $\sum (\text{flow} \times dt)$ with `dt` in **minutes**, giving a mass in **grams**; `fm101_total` therefore has unit `g` . Grafana may display kilograms, but the stored value is grams.

Two traps, both covered explicitly in `test_scaling.py` : a `dt` in seconds produces a factor-60 error that looks entirely plausible, and the LabVIEW front panel labels this channel "Flow (SLPM)", which is wrong — it is not a volumetric standard-litre flow.

8. Interfaces and access

Hard requirement: the system is reachable from the lab PC only. Every listening service binds to the loopback address. This matches how the LabVIEW system works today and is a deliberate constraint, not an oversight to be relaxed later without discussion.

```
uvicorn.run(app, host="127.0.0.1", port=8000) # NOT 0.0.0.0
```

`0.0.0.0` is the default in most tutorials and in much example code. On this system it would expose HV control to the building network. Every bind address is therefore explicit in configuration, and `127.0.0.1` is the only accepted value without a recorded decision to the contrary.

The same applies to everything else that opens a port:

Service	Bind	Note
FastAPI web UI	127.0.0.1:8000	control surface
Mosquitto	127.0.0.1:1883	see below

Service	Bind	Note
PostgreSQL	127.0.0.1:5432	
Grafana	127.0.0.1:3000	

Mosquitto deserves particular care. Commands travel over MQTT (`xams/cmd/#`), so anything that can reach the broker can set a high voltage. A broker listening on all interfaces is equivalent to an unauthenticated control interface on the network. Bind to loopback, and disable anonymous access on any listener that is ever added.

If remote viewing is wanted later, the answer is an SSH tunnel or a read-only mirror of the data — never opening the control port. Adding remote access is a decision with a security consequence, and should be recorded as such.

8.1 The four surfaces

Separated by how often they are touched and how much a mistake costs.

Grafana — monitoring. Plots, history, dashboards, trends. Read-only by construction, and where most of the day is spent. It **displays** alarm state and alarm history, but it does not evaluate alarms or send notifications — that is `alarms/engine.py` (§11). No control, ever: that separation is the point.

Web UI — the landing page. `http://localhost:8000/` is the page to bookmark and open every morning. It answers "is everything all right?" without a click, and only then offers links.

XAMS Slow Control ● ALL OK config a3f91c2 · v1.4.0 · up 6d 4h	
Services	cdaq ● 2s caen ● 3s lakeshore ● 2s ups ● 11s
Sinks	postgres ● 1s jsonl ● 1s nikhef-vm ▲ 4m behind
Alarms	none active
Disk	C: 812 GB free
Grafana · Status · Recipients · Logs	

Page	Contents
/	the overview above, self-refreshing
/status	per channel: value, unit, age, quality
/recipients	edit the notification list (§4.4)
/logs	the last lines of each service log — saves logging in and hunting for files
/control	setpoints and procedures — built last (§10)

Plus links to Grafana on `:3000` and to `DESIGN.md` and `OPERATIONS.md` in the repository.

The status page reads from MQTT retained topics, never from the database. If PostgreSQL is down the page must still work — that is precisely when it is needed. A status page that fails together with the component it reports on is worthless.

Kept deliberately plain: server-rendered HTML from FastAPI, a meta refresh or a few lines of `fetch`, no JavaScript framework and no build step. In three years a student must be able to change it without installing a toolchain.

Web UI — control. Current value of every channel, alarm state, service health, and the control actions. Per HV channel it shows `VSET`, `VMON`, `IMON`, on/off state and ramping status.

Setting a value requires a confirmation step. For HV the new value is retyped rather than confirmed with a click — deliberate friction, because the failure mode is an extra zero on an electrode. Validation before any write: identity

confirmed (§6.2), value within the range in `channels.yaml`, device not in an error state. A rejected command is acknowledged with a reason, never silently dropped.

The page also carries alarm **acknowledge** (stop the repeating notification; the condition is still displayed as active) and **silence for N minutes** (for a deliberate intervention). Acknowledging is not fixing, and the UI must never let the two look alike.

Config files in git — the system's definition. Calibrations, channel map, alarm thresholds. Edited in a text editor, committed, then applied without a restart and without a gap in the data:

```
xams-ctl reload
```

These are not editable from the web UI — with one exception, `recipients.yaml` (§4.4), which changes often and carries no safety consequence. A wrong threshold silently disables protection and is discovered months later; in git it has review, history, and the config hash in every data file records exactly which version produced which data (§4.4). A clickable threshold has none of that.

The honest cost: changing a limit means editing a file rather than dragging a slider. For a handful of changes a year that is the right trade. If thresholds turn out to need weekly adjustment, the fix is a UI that edits the YAML **and** commits it — the same audit trail with less friction. Build that only once the need is demonstrated.

Python client — scripts and notebooks.

```
from xams_sc import Client
c = Client()
c.get("pmain")                # current value
c.history("tt201", hours=24)  # pandas DataFrame
c.set("hv_anode", 3500)       # same validation, same audit log as the UI
```

This is what makes the system useful beyond monitoring: calibration runs, scripted ramps, correlation studies. It is also how the `anode_timing.vi` behaviour would be reimplemented if it turns out to be needed (§7.2).

For debugging, `mosquitto_sub -t 'xams/#'` shows everything live with no UI involved.

8.2 What is deliberately not in any interface

Ramp rates, trip currents, over-voltage limits, heater range and setpoint limits live on the instruments themselves. The software **displays** them and raises an alarm if they differ from the expected values in `devices.yaml`, but it cannot change them. This is what keeps the software out of the protection path (§10).

9. Storage

What exists today. The CSV files on the lab PC are the authoritative record; the MongoDB instance on the Nikhef server is temporary storage only, and is retired when LabVIEW is. So there is currently **no permanent archive** — the entire history sits on one disk, in a format whose column meanings live in a separate, overwriteable header file.

Creating a durable archive is therefore not a reorganisation of something existing. It is the single largest improvement in this design.

Three layers, each with one job.

9.1 Sampling and logging rates

Sampling rate and logging rate are separate decisions. Conflating them is why the LabVIEW system writes a record every couple of seconds for channels whose physics moves in minutes.

```
sampling:
  interval_s: 1          # read the hardware
  log_interval_s: 10     # write the mean of those samples
  flight_recorder:
    window_s: 600        # last 10 minutes at 1 Hz, held in memory
    dump_on: [alarm, trip, service_restart]
```

Read at 1 Hz, log every 10 s as the mean of the ten samples. The averaging is not merely data reduction — a mean of ten readings is less noisy than any single one, so the stored value is better than what is discarded.

Disk is not the constraint. At 30 channels and 1 Hz the cost would be roughly 10 MB gzipped per day, which is nothing on this machine. The reasons to log less often are query speed, plot legibility and ease of analysis. At 10 s the system produces about 260 000 records per day, ~1 MB gzipped, ~400 MB per year: ten years of history in 4 GB.

Why 10 s and not slower. The physics differs per channel:

Channel	Time constant	1 Hz is
Cryostat RTDs	minutes	heavily oversampled
Ambient temperature	tens of minutes	absurdly oversampled
Pressure, steady state	minutes	oversampled
Pressure, while pumping or filling	seconds	about right
HV VMON / IMON , normal operation	steady	oversampled
HV during a trip	milliseconds	far too slow

A fixed 1 Hz is therefore simultaneously too fast for the temperatures and too slow for the only moment that resolution matters. Temperatures alone would be well served by 60 s, but **the rate is deliberately uniform across all channels**: identical timestamps let channels be compared without interpolation, and that simplicity is worth more than the few megabytes differentiation would save.

Flight recorder. The last 10 minutes at the full 1 Hz are held in a fixed-length `collections.deque`. When an alarm fires, a channel trips, or a service restarts unexpectedly, the buffer is written out:

```
data/events/2026-09-16T13-02-11_pmain_hihi.jsonl
```

This gives full resolution exactly around the moments that matter, without storing it permanently. It is what the present system lacks: after an incident there are only averaged values, and no record of the approach to it.

Optionally, for pressure and HV channels, store `min` and `max` of each interval alongside the mean, so short excursions are not lost between log points. Two extra columns; decide per channel in `channels.yaml`.

9.2 PostgreSQL — working store

PostgreSQL. Not InfluxDB, not "Timescale or Postgres" — TimescaleDB is an *extension* to PostgreSQL, not a separate product, and at this data volume it is not needed. It can be enabled later on the same database, without touching the data, if compression and automatic retention become worth having.

Rationale: a native Windows installer, a built-in Grafana datasource needing no plugin, `psycopg` and `pandas.read_sql` in Python, and SQL that anyone in the group can read today and in ten years. InfluxDB was the obvious default next to Grafana, but its Windows support is uneven and its data model and query language have been reworked twice between major versions — the wrong kind of churn for a system meant to be installed and left alone.

One table:

```
CREATE TABLE meas (
  t          timestampz NOT NULL,
  channel    text       NOT NULL,
  value      double precision,      -- mean over the log interval
  vmin       double precision,      -- optional, per channel
  vmax       double precision,      -- optional, per channel
  raw        double precision,
  unit       text,
  quality     text          NOT NULL DEFAULT 'ok',
  src         text          NOT NULL DEFAULT 'xams' -- 'labview' for imported history
);
CREATE INDEX ON meas (channel, t DESC);
```

Written by `sinks/pg_writer.py`, subscribed to `xams/meas/#`. Batched, flushed at least every 10 s. **A failure to write must never block the bus or the services** — the writer logs, alarms, and keeps going.

This database is a **cache, not storage**. The archive is the files (§9.3, §9.4). If the database is lost or a different one is wanted later, write a new sink and replay the Parquet history into it. Nothing in any driver changes.

Grafana reads from here.

9.3 JSONL — raw archive

One file per UTC day: `data/raw/2026-09-16.jsonl`. Written by `sinks/jsonl_writer.py`, **independently of the database**. If the database falls over, no data is lost. The files are the truth; the database is an index over them.

First line of every file:

```
{"t":"2026-09-16T00:00:00Z","meta":{"config":"a3f91c2","version":"1.4.0","host":"xams-sc"}}
```

Flush and `fsync` at least every 10 s so a power loss costs seconds, not hours.

9.4 Parquet — long term

`tools/jsonl_to_parquet.py` runs nightly on the closed day, then gzips the JSONL. Both are kept until the conversion has been verified; after that, keeping the gzipped JSONL as well is cheap and worth it.

Log raw alongside the scaled value for at least the first year. If a multiplier turns out to be wrong — the `×25` on `p101` is a candidate — the entire history can be rescaled. Without `raw`, it cannot. At ~30 channels every 2 s this costs roughly 120 MB/day uncompressed, around 10 MB gzipped.

9.5 MongoDB — temporary, during the transition

The existing MongoDB writer can be reused as a third sink, so the Nikhef server keeps receiving data and the existing Python viewer keeps working while Grafana is being set up. Adding it touches no driver: it is another MQTT subscriber.

Deliberately a compatibility layer with an expiry. It initially emits the current wide document shape, so the existing viewer needs no changes. Once Grafana is running it either moves to the long form or is removed. Record that expiry here and in the code — an undated temporary layer becomes permanent, which is how the present system came about.

Same rules as the other sinks: a Mongo failure never blocks the bus, and a TTL index on the timestamp keeps the collection from growing without bound. MongoDB is a convenience copy, never a second source of truth.

9.6 Importing the LabVIEW history

`tools/import_labview_csv.py` converts the existing CSV history into the same JSONL/Parquet form, so Grafana shows one continuous timeline across the migration rather than starting from zero. This is cheap and worth doing.

One hazard, which must be checked before the import is trusted. The LabVIEW front panel has a "Re-save headers" button, so the header file is mutable and there may be only one — describing only the most recent channel layout. If a channel was ever added, removed or reordered, older CSV files silently do not match it. Columns shift by one position and the values still look entirely plausible.

Check first:

```
for f in *.csv; do printf "%s %s\n" "$(head -1 "$f" | awk -F';' '{print NF}')" "$f"; done | sort | uniq -c -w4
```

One group of column counts means one layout throughout, and the import is straightforward. Several groups mean several layouts, and each era needs its own column mapping — recovered by hand, from file dates and from what the values physically are. The importer must refuse to guess: a file whose column count does not match its declared mapping is rejected, not imported with a warning.

Imported records are tagged `"src":"labview"` so they are distinguishable from data taken by the new system.

10. Control path and safety

The slow control is not a protection system. It monitors, logs, alarms, and offers convenience setpoint changes.

Rules

1. **Interlocks live in hardware.** HV trip on a pressure or vacuum excursion is wired to the CAEN interlock input from a real gauge trip. Not a Python service reading MQTT and deciding.
2. **Limits live in the instrument.** Ramp rate, trip current, over-voltage limit, heater range, setpoint limits are configured on the device. The software **verifies** them at startup and alarms on mismatch; it does not write them.
3. **No write without verified identity** (§6.2).
4. **Restart never actuates** (§6.1).
5. **Every write is logged** — who, what, when, previous value, new value — to an append-only audit file and to `xams/ack/#`.

Command flow

```
UI / API → xams/cmd/<device>/<action> → service validates → instrument
                                     ↓
                                     xams/ack/<device>/<action>
```

Validation before any write: identity confirmed, value within the configured range, device not in an error state. A rejected command is acknowledged with a reason; it is never silently dropped.

Procedures

Operating the Lake Shore and the CAEN supplies by hand is tedious: menu navigation on the front panel for a setpoint, per-channel navigation across two supplies for high voltage. Bringing the TPC up means eight manual steps in the right order, which is exactly the kind of work that goes wrong at two in the morning.

Control therefore does more than save effort — a written, reviewed sequence that always proceeds in the same order is **more reliable than a person with a printed procedure**. This is an argument for building the control path, not merely a convenience.

Named sequences live in git as data, like everything else:

```
# procedures/hv_rampup.yaml
name: HV ramp up
steps:
  - set: {channel: hv_cathode, value: -10000, ramp: 50}
  - wait_until: {channel: hv_cathode, stable_within: 10, timeout_s: 600}
  - set: {channel: hv_gate, value: -5000, ramp: 50}
abort_on: [any_alarm, trip, operator_stop]
```

Requirements:

- **Dry run.** Every procedure can be executed in a mode that reports each step without writing anything. This is also how a procedure is reviewed before it is run for the first time.
- **Abort is always available**, and aborting leaves the hardware in a defined state — it does not simply stop mid-sequence without saying where it stopped.
- **Every step passes the ordinary write validation** (§6.2, and the rules above). A procedure is not a way around the guardrails.
- **The whole run is audited**, step by step, not merely as "procedure executed".
- A procedure that is edited is a change to a file in git, reviewable like code.

Open decision

The existing LabVIEW system performs protective actions in software: a "Shut off heater if Alarm is active" switch and a `KILL VOLTAGE` button. **TBD** — reproduce as-is, or move into hardware. Reproducing it is defensible; doing so without noticing the choice is not. Decide before the control path is implemented.

11. Alarm engine

`alarms/engine.py` subscribes to `xams/meas/#` and evaluates against `alarms.yaml`. It decides and notifies; Grafana only displays the result.

Grafana's own alerting is deliberately not used. It works by querying the database on a schedule, which places PostgreSQL and Grafana inside the alarm path: if either is down or slow, notifications do not go out. Alarms are the part of this system that must be most reliable, so the path is kept as short as possible — measurement → bus → engine → SMS, with no database and no web server involved. Grafana also has no native SMS contact point, so a script would have to be written regardless.

The engine publishes alarm state to `xams/alarm/<channel>`, which is stored like any other record, so Grafana can show current state and history without being part of the mechanism.

Thresholds live in one place

Displaying alarm *state* needs no knowledge of the limits: the engine publishes a result (`{"state": "major"}`) and Grafana colours a panel from it. Drawing a threshold *line* on a graph is different — Grafana would need the number, and it would then exist both in `alarms.yaml` and in the dashboard JSON, where the two will drift apart.

Therefore: no threshold lines on graphs. The alarm panel colours, which is what is actually watched. Generating the dashboard JSON from `alarms.yaml` would solve it properly, but the cost is not the generator (a template substitution, some forty lines) — it is the recurring workflow: every dashboard edited in the Grafana UI must be re-exported and re-templated, or the generator overwrites the manual work. For a handful of panels that is more friction than the problem warrants. Revisit it if the lines turn out to be genuinely missed *and* have drifted once.

Instead, make drift visible. On startup and on every reload the engine publishes the limits it actually loaded:

```
xams/status/limits {"pmain": {"high": 1.8, "hihi": 2.0}, ...}
```

shown on the status page and written to the log. The real risk is not a missing line on a plot; it is believing a threshold is 2.0 when it is 20. This exposes what is in force, with no generation machinery.

- Four thresholds per channel, EPICS-style: `lo`, `lo`, `low`, `high`, `hihi`, each with a severity.
- **Hysteresis** on every threshold, to stop a channel sitting on a limit from producing a stream of notifications.
- **Deduplication**: one notification per state transition, then repeat at `min_repeat_minutes` while the condition persists.
- **Staleness is an alarm**, at the same severity as a threshold breach. A dead sensor must not read as healthy.
- Alarm state is published on `xams/alarm/<channel>` and is itself retained, so the UI shows the true state on connect.

Notification

`alarms/notify.py` exposes a thin interface — `send_sms(number, text)`, `send_email(...)` — and the alarm engine knows nothing else, so what sits underneath is replaceable without touching alarm logic. Recipients come from `recipients.yaml` (§4.4); severity routing from `alarms.yaml`.

Reuse the existing SMS script. LabVIEW already calls a Python script (`SC_software\send_sms_python\`); the new system can import it as a module instead of starting a subprocess. The argument is risk, not convenience: that script works. The numbers are right, the gateway accepts it, the message format arrives, the costs are known. Rebuilding it means rediscovering all of that. The same applies to the email script and `check_ups.vi`.

Three things to check before adopting it: whether it is Python 2 or 3; whether phone numbers are hardcoded (they belong in `recipients.yaml` now, passed in as arguments); and whether it writes a status file that only LabVIEW reads.

Credentials never enter the repository. An API key committed to git remains in the history after deletion, and private repositories are still cloned, shared and backed up. Credentials live in `config/secrets.yaml`, listed in `.gitignore`; `config/secrets.example.yaml` is committed with empty values so the required keys are documented. `notify.py` reads from there, never from code.

While examining the existing script, check whether its key is still valid and who else holds it. A gateway credential that has sat on a desktop for years is a good candidate for rotation.

12. Operations

Service management

Each service runs under NSSM as a Windows service, with stdout/stderr to `logs/<service>.log`, restart on exit after 5 s.

Automatic start stays off while LabVIEW is the fallback. Every device admits only one process, so a service that auto-starts after an overnight reboot will claim the hardware and lock LabVIEW out.

Phase	Start type	Notes
Debug	not installed	run from a terminal; let it die on errors
Trial	<code>SERVICE_DEMAND_START</code>	restarts on crash, not at boot
Production	<code>SERVICE_AUTO_START</code>	only after LabVIEW is retired

`xams-ctl`

```
xams-ctl start | stop | restart | status | reload
```

`reload` re-reads the YAML configuration without restarting the services, so a threshold or calibration change costs no gap in the data.

`stop` releases all hardware for LabVIEW. `status` shows each service's state and heartbeat age. One command, correct order, every time.

Alarm watchdog on the VM

The alarm engine cannot report that the alarm engine has stopped, and the heartbeat monitor (§6.3) runs in the same Python environment on the same machine — if that goes down thoroughly, both are silent.

If a Nikhef VM exists, configure **one** Grafana alert rule there:

```
no measurement received for 15 minutes → email
```

A dead-man's switch with a completely separate code path, on a different machine, on a different network. It catches what nothing local can: the lab PC off or crashed, Windows rebooting for updates, services stopped and not restarted, or the network between lab and Nikhef down.

This is a single rule about silence, not a physics threshold, so it duplicates nothing from `alarms.yaml`. A second copy of this rule on the local Grafana would be pointless: if the PC is off, that Grafana is off too.

It is also the strongest argument for setting up the VM — stronger than colleagues being able to look at plots. Without an outside observer, "the system has gone completely quiet" is the one failure that cannot be detected.

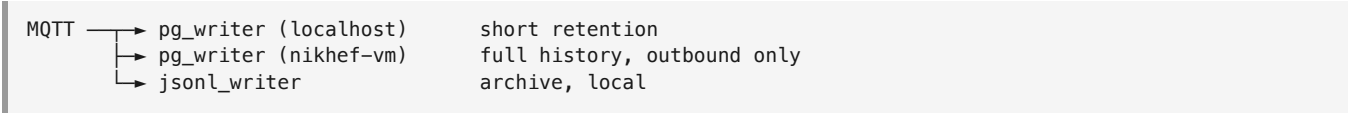
Deployment and topology

The lab PC is self-contained: everything needed to acquire, store, plot and alarm runs there, and none of it depends on any other machine. A Nikhef VM, if one is added later, is a copy plus a window for the rest of the group.

Component	Lab PC	Nikhef VM (optional)
Device services	✓	—
Mosquitto	✓	—
Alarm engine	✓ always	—
JSONL + Parquet archive	✓ the truth	copy
PostgreSQL	✓ short retention (~30 days)	✓ full history
Grafana	✓ a few operational dashboards	✓ the full set, for the group
Web UI / status page	✓	—

The alarm engine runs locally and never depends on the VM. When the network is down is exactly when the operator needs to see what is happening, so the local view must stand on its own.

Two writers, no added complexity — the same sink with a different address:



If the VM is unreachable only that one writer fails; it buffers and catches up, and nothing local notices.

Reproducible installation

`sql/schema.sql` and the provisioned Grafana dashboards exist regardless of whether a VM is ever set up, because they make the **local** install reproducible. Grafana stores dashboards in its own internal database: without provisioning they are lost when that database is lost or Grafana is reinstalled, taking hours of work with them. With the schema and the dashboards in git, installation is a procedure rather than something that was once assembled by hand and can no longer be repeated. This is the same argument as for `channels.yaml`: configuration is data, not an action living in someone's memory.

Installing anywhere — lab PC or VM — is then:

```
git clone <repo> /opt/xams-sc
psql -U postgres -d xams -f /opt/xams-sc/sql/schema.sql
# point Grafana provisioning at /opt/xams-sc/grafana/
```

and updating is `git pull`. A dashboard edited on the lab PC, exported to JSON and committed, appears on the VM at the next pull. No copying, no versions drifting apart, no deployment tooling.

`secrets.yaml` is in `.gitignore`, so it never travels with the repository; each machine keeps its own.

Logging

Python `logging` to rotating files, INFO by default, DEBUG selectable per service. Every log line carries the service name. Hardware errors log the raw instrument response, not just the parsed exception — that is what makes protocol bugs findable.

13. Testing

Simulation mode. Every device class accepts `simulate=True` and returns plausible synthetic data without hardware. The whole stack — bus, writers, alarms, UI — must be runnable on a laptop with no instruments attached. This is what makes development possible without occupying the lab PC.

Unit tests, no hardware required:

- `test_scaling.py` — `(raw - offset) * multiplier`, including the recovered values for `p101`, `pmain`, `fm101`
- `test_config.py` — schema validation, duplicate channel names, unknown device references, config hash stability
- `test_caen_protocol.py` — response parsing against recorded strings, including malformed and truncated replies

Hardware checklist, run once per device on first connection:

1. Identity query returns the expected serial number
 2. All configured channels read without error
 3. Values are physically plausible
 4. Disconnecting the device produces `quality=error` and a reconnect attempt, not a crash
 5. Reconnecting recovers without a restart
 6. Restarting the service changes nothing on the instrument
-

14. Documentation

Documentation is written **during** construction, one increment per milestone — not afterwards. Written at the end it records what the author remembers rather than what a reader does not understand, and by then everything has become obvious to the person who built it. That is how the LabVIEW project ended up without usable documentation, and it was not for lack of goodwill.

All of it lives in the repository, in git, beside the code.

Document	Audience	Written
<code>README.md</code>	someone reinstalling the system from scratch	milestone 1
<code>OPERATIONS.md</code>	the group, daily	grows with each milestone
<code>DESIGN.md</code>	someone changing the system	this document
Docstrings, and <code>description</code> in <code>channels.yaml</code>	someone reading the code or the config	as written

`OPERATIONS.md`

The most important of the four, and the one most often missing. Not architecture — actions:

- starting, stopping, checking status
- **what each alarm means and what to do about it**, per threshold
- replacing a sensor and adjusting its calibration
- adding a channel
- what to do when a service will not start, when a CAEN supply stops responding, when there is a gap in the data

- how to restore LabVIEW if the system has to be rolled back

The alarm point is not hypothetical. The thresholds of the present system exist in the VIs, but the *response* to each exists only in people's heads (§16). A threshold without a prescribed action is half the information, and reproducing that omission would waste the migration.

The troubleshooting section grows: every time something breaks and is fixed, it goes in. After a year it is the most valuable file in the repository.

Acceptance

Documentation is not accepted because its author considers it complete. The test is that a colleague, using `OPERATIONS.md` alone, can stop the system, add a channel and start it again — without asking the author anything.

If that fails, the document is unfinished however thorough it looks. This is also the only real mitigation for the bus-factor problem: a manual nobody has ever used proves nothing.

15. Milestones

Each milestone has an acceptance criterion. Do not start the next before the current one passes.

#	Milestone	Acceptance criterion
1	Skeleton	<code>BaseService</code> , config loading, MQTT bus, simulation mode. Fake service publishes, writers store, Grafana plots. No hardware. <code>README.md</code> lets someone else reproduce the install, and the install is scripted: <code>sql/schema.sql</code> plus provisioned Grafana dashboards.
2	cDAQ read-only	All connected channels (6 voltage + 14 RTD) read and logged, LabVIEW stopped. Values plausible. Restart changes nothing.
3	Channel verification	Each <code>tt*</code> tag confirmed empirically against its channel (warm a sensor, watch which value moves) and its physical location recorded. Tags are known; the mapping to hardware is what is being verified.
4	Scaling and history	Column-count check passed (§9.6), history imported, and scaled values agree with the LabVIEW record for the same sensors within expected tolerance.
5	Lake Shore + CAEN monitoring	Read-only. Identity verification working. Unplug test passes.
6	UPS, alarms, flow integrator	Thresholds from <code>alarms.yaml</code> , SMS and email delivered, staleness alarms fire, flight-recorder dump produced on a test alarm. Integrator survives a service restart without losing its total.
7	Web UI	Current values, alarm state, service health. Read-only, bound to <code>127.0.0.1</code> . Python client works from a notebook.
8	Control path	Lake Shore setpoint, then HV — only after §10's open decision is made and monitoring has run reliably for weeks. Audit log complete; every write validated; dry run works.
9	Procedures	Named sequences (§10) run, abort cleanly, and are audited step by step.
10	Production	<code>SERVICE_AUTO_START</code> , LabVIEW retired but installed. <code>OPERATIONS.md</code> passes the acceptance test of §14 — a colleague operates the system from it unaided.

Milestone 3 is the one that cannot be rushed. The tag names are now known, but a tag that was already attached to the wrong channel in the LabVIEW system would be copied across silently. Verifying each one physically is the only way to catch that.

16. Open items

Everything marked **TBD** above, consolidated:

Item	Needed for	How to resolve
Physical location of each <code>tt*</code> tag, and what the 1xx/2xx/3xx series denote	milestone 3	lab knowledge; record in <code>description</code>
Alarm thresholds and responses	milestone 6	<code>Error</code> and <code>Alarm</code> tab screenshot
HV channel → supply/index mapping	milestone 5	<code>High Voltage Supply</code> tab screenshot
HV setpoints, ramp rates, trip limits	milestone 8	LabVIEW tab + settings stored on the CAEN units
USB serial numbers of the two CAEN units	milestone 5	<code>list_ports</code> on the lab PC
Lake Shore baud rate	milestone 5	instrument front panel
UPS model and connection	milestone 6	inspect the unit
Are <code>9207/ai8:15</code> current channels used?	milestone 2	inspect wiring
Engineering units for the pressure channels (<code>p101</code> – <code>p104</code> , <code>pmain</code>)	milestone 4	lab knowledge, or the transducer datasheets
Purpose of <code>anode_timing.vi</code> , <code>DAISY_polarity_signs.vi</code>	milestone 8	read the block diagrams
Heater shut-off and HV kill: hardware or software?	milestone 8	decision
How far back the CSV history goes, and whether the column count is constant throughout	milestone 4	the check in §9.6
Second maintainer	production	decision